



TP 4 – Écran de détail produit

Composants fondamentaux

1. Objectif du TP

Dans ce TP, vous allez démarrer une nouvelle application Compose qui servira de base aux prochains TP du bloc 2. Cette application s'appellera : **Product Explorer**.

Elle permettra progressivement d'afficher une interface de catalogue produit.

Dans ce premier TP, vous allez créer un **écran de détail produit**.

L'objectif est de manipuler les **composants fondamentaux** de Jetpack Compose dans un cas concret.

Vous allez utiliser : **Text** ; **Button** ; **Icon** ; **Image** ; **Card** ; **TextField**.

À la fin du TP, vous aurez un écran qui affiche : *une **image** produit ainsi que **titre**, **catégorie**, **marque**, **prix**, **remise**, **note**, **stock**, **description**, **information complémentaire**, **bouton** d'action.*

2. Cadre du TP

Dans ce TP, les données sont **locales**. Cela signifie que le produit affiché sera créé directement dans le code Kotlin. Nous ne récupérons pas encore les données depuis Internet.

Le TP reste centré sur les **composants visuels**. Le bouton ne déclenche pas encore une vraie action métier.

3. Créer le projet Product Explorer

Créez un nouveau projet Android Studio.

- Choisissez un projet Compose vide.
- Nom possible du projet : **ProductExplorer**
- Nom possible du package : **com.example.productexplorer**
- Une fois le projet créé, ouvrez le fichier **MainActivity.kt**.

Vous devriez avoir une structure proche de celle-ci :

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ProductExplorerTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    // Contenu de l'écran
                }
            }
        }
    }
}
```

Nous avons volontairement supprimé le composable par défaut **Greeting**, ainsi que son appel depuis le **Scaffold** et sa **@Preview**.

Le nom du **thème** dépend du nom de votre projet.

Si votre thème ne s'appelle pas **ProductExplorerTheme**, gardez le nom généré par Android Studio.



4. Étape 1 – Préparer une donnée produit locale

Avant de construire l'interface, nous avons besoin d'un **produit à afficher**. Pour cela, nous allons créer une **classe simple**.

Cette classe représente les informations nécessaires à l'écran de détail. Elle ne fait aucun appel réseau. Elle sert seulement à **transporter** des données vers l'interface.

Dans **MainActivity.kt**, ajoutez cette **classe** en dehors de **MainActivity** :

```
data class ProductUi(  
    val title: String,  
    val brand: String,  
    val category: String,  
    val description: String,  
    val price: Double,  
    val discountPercentage: Double,  
    val rating: Double,  
    val stock: Int,  
    val warrantyInformation: String,  
    val shippingInformation: String  
)
```

Le suffixe **Ui** indique que cette classe est utilisée pour afficher des informations dans **l'interface**.

Pour l'instant, elle contient directement les données dont l'écran a besoin. Plus tard, dans le cours, les données pourront venir d'une autre source. Mais ici, elles restent locales.

5. Étape 2 – Créer un exemple de produit

Nous allons maintenant créer un **produit d'exemple**. Ce produit sera utilisé par notre écran principal.

L'intérêt est de travailler avec des données réalistes, sans dépendre d'un serveur.

Ajoutez cette fonction sous la classe **ProductUi** :

```
fun sampleProduct(): ProductUi {  
    return ProductUi(  
        title = "Smartphone Toto X",  
        brand = "TotoTech",  
        category = "Smartphones",  
        description = "Un smartphone léger avec un écran lumineux, une bonne autonomie et un design moderne.",  
        price = 699.99,  
        discountPercentage = 12.5,  
        rating = 4.6,  
        stock = 34,  
        warrantyInformation = "Garantie constructeur : 2 ans",  
        shippingInformation = "Livraison estimée : 3 à 5 jours ouvrés"  
    )  
}
```

Cette fonction retourne toujours le même produit.

C'est suffisant pour construire notre écran de détail.

Nous cherchons d'abord à savoir comment afficher correctement les informations.

6. Étape 3 – Afficher l'écran principal

Dans **MainActivity**, nous allons appeler un composable nommé **ProductDetailScreen**. Ce composable sera responsable de l'affichage du détail produit.

Il recevra : **un produit**, **un modifier** et **une action de clic** pour le bouton.

Modifiez le contenu de **Scaffold** :



```
Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
    ProductDetailScreen(
        product = sampleProduct(),
        onAddToCartClick = {
            // Action à ajouter plus tard
        },
        modifier = Modifier.padding(innerPadding)
    )
}
```

À ce stade, **ProductDetailScreen** n'existe pas encore. C'est normal. Nous allons le créer dans l'étape suivante.

7. Étape 4 – Créer le composable *ProductDetailScreen*

ProductDetailScreen représente l'écran complet.

Pour commencer, nous allons afficher les éléments les uns sous les autres dans une **Column**.

Ici, nous utilisons déjà plusieurs composants :

- **Text** pour afficher les informations ;
- **Button** pour l'action ;
- **Column** pour organiser l'écran verticalement.

Voici donc le code de composable **ProductDetailScreen** à ajouter :

```
@Composable
fun ProductDetailScreen(
    product: ProductUi,
    onAddToCartClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        Text(text = product.title)
        Text(text = product.brand)
        Text(text = product.category)
        Text(text = "${product.price} €")
        Text(text = product.description)

        Button(onClick = onAddToCartClick) {
            Text(text = "Ajouter au panier")
        }
    }
}
```

Ce premier écran est très simple : il affiche bien les **données**, mais tout est encore brut.

Pour l'instant, ce n'est pas grave, nous allons améliorer progressivement l'écran en créant des composants plus lisibles.

Astuce : si vous ne copiez-collez pas le code mais que vous l'écrivez vous-même en utilisant l'auto-complétions, Android Studio ajoute progressivement les importations nécessaires automatiquement en haut du fichier.

8. Étape 5 – Ajouter une *preview* de l'écran

Une **preview** permet de vérifier rapidement le rendu d'un composable sans lancer toute l'application. Nous allons créer une **preview** pour **ProductDetailScreen**.

Ajoutez cette **preview** :



```
@Preview(showBackground = true)
@Composable
fun ProductDetailScreenPreview() {
    ProductExplorerTheme {
        ProductDetailScreen(
            product = sampleProduct(),
            onAddToCartClick = {}
        )
    }
}
```

Si votre thème ne s'appelle pas `ProductExplorerTheme`, remplacez ce nom par celui généré dans votre projet.

Cette *preview* utilise le même produit d'exemple que l'application.

Le callback `onAddToCartClick` est vide, car nous ne voulons pas encore exécuter une vraie action.

9. Étape 6 – Extraire l'en-tête du produit

Actuellement, `ProductDetailScreen` contient trop d'informations directement.

Nous allons extraire la **partie haute** de l'écran dans un composable séparé.

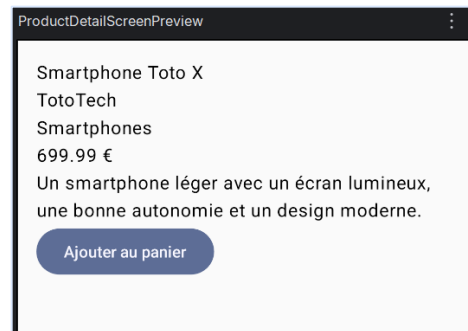
Cet **en-tête** affichera :

- le **nom** du produit ;
- la **marque** ;
- la **catégorie**.

Cela rendra le code plus lisible.

Ajoutez ce composable :

```
@Composable
fun ProductHeader(
    title: String,
    brand: String,
    category: String,
    modifier: Modifier = Modifier
) {
    Column(modifier = modifier) {
        Text(text = title)
        Text(text = brand)
        Text(text = category)
    }
}
```



Puis modifiez `ProductDetailScreen` :

```
@Composable
fun ProductDetailScreen(
    product: ProductUi,
    onAddToCartClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
        .fillMaxSize()
        .padding(16.dp)
    ) {
        ProductHeader(
            title = product.title,
            brand = product.brand,
            category = product.category
        )

        Text(text = "${product.price} €")
        Text(text = product.description)
    }
}
```



```
Button(onClick = onAddToCartClick) {  
    Text(text = "Ajouter au panier")  
}  
}
```

ProductHeader reçoit maintenant seulement les informations dont il a besoin. Il ne reçoit pas tout l'objet **ProductUi**. Cela rend le composable plus clair et plus réutilisable.

10. Étape 7 – Améliorer les textes avec *MaterialTheme*

Text permet d'afficher du **texte**. Mais tous les textes n'ont pas la même importance. Un **titre** doit être **plus visible** qu'une **information secondaire**.

Compose permet d'utiliser la typographie du **thème** avec **MaterialTheme.typography**.

Nous allons améliorer l'affichage de l'en-tête.

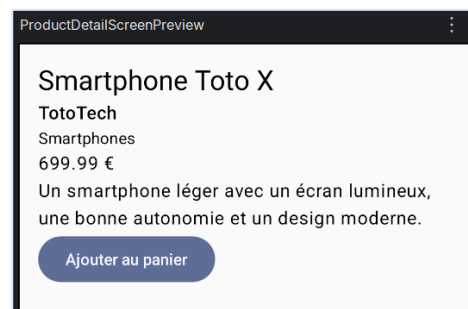
Modifiez **ProductHeader** :

```
@Composable  
fun ProductHeader(  
    title: String,  
    brand: String,  
    category: String,  
    modifier: Modifier = Modifier  
) {  
    Column(modifier = modifier) {  
        Text(  
            text = title,  
            style = MaterialTheme.typography.headlineSmall  
        )  
        Text(  
            text = brand,  
            style = MaterialTheme.typography.titleMedium  
        )  
        Text(  
            text = category,  
            style = MaterialTheme.typography.bodyMedium  
        )  
    }  
}
```

Nous utilisons ici **trois styles** différents :

- **headlineSmall** pour le **titre** ;
- **titleMedium** pour la **marque** ;
- **bodyMedium** pour la **catégorie**.

Le contenu devient plus lisible sans ajouter beaucoup de code.



11. Étape 8 – Créer une carte pour le prix

Le **prix** est une information importante. Nous allons donc l'afficher dans une **Card**.

Une **Card** permet de **regrouper des informations** dans un bloc visuel identifiable.

Dans notre cas, la carte contiendra le **prix** et la **remise**.

Ajoutez ce composable :



```
@Composable
fun ProductPriceCard(
    price: Double,
    discountPercentage: Double,
    modifier: Modifier = Modifier
) {
    Card(modifier = modifier) {
        Column(
            modifier = Modifier.padding(16.dp)
        ) {
            Text(
                text = "$price €",
                style = MaterialTheme.typography.headlineSmall
            )
            Text(
                text = "Remise : $discountPercentage %",
                style = MaterialTheme.typography.bodyMedium
            )
        }
    }
}
```

Puis utilisez-le dans `ProductDetailScreen` :

```
ProductPriceCard(
    price = product.price,
    discountPercentage = product.discountPercentage,
    modifier = Modifier
        .fillMaxWidth()
        .padding(top = 16.dp)
)
```

Placez cet appel juste après `ProductHeader` à la place de `Text(text = "${product.price} €")`.

- La `Card` rend l'information plus visible.
- Le **padding interne** de `16.dp` espace le contenu à l'intérieur de la carte.
- Le `Modifier` passé depuis `ProductDetailScreen` contrôle la place de la carte dans l'écran.

12. Étape 9 – Ajouter une image produit

Nous allons maintenant afficher une **image**.

Dans ce TP, nous utilisons une **image placeholder** fournie par Android.

Cela permet d'apprendre à utiliser `Image` sans gérer de téléchargement d'image.

Nous utiliserons :

- `Image` pour afficher l'image ;
- `painterResource` pour charger une ressource ;
- `contentDescription` pour décrire l'image.

Ajoutez ce composable :

```
@Composable
fun ProductImage(
    productTitle: String,
    modifier: Modifier = Modifier
) {
    Image(
        painter = painterResource(id = android.R.drawable.ic_menu_gallery),
        contentDescription = "Image du produit $productTitle",
        modifier = modifier
    )
}
```

Puis ajoutez-le au début de `ProductDetailScreen`, avant `ProductHeader` :



```
ProductImage(  
    productTitle = product.title,  
    modifier = Modifier  
        .fillMaxWidth()  
        .padding(bottom = 16.dp)  
)
```

- **Image** affiche un élément visuel.
- **contentDescription** décrit l'image pour les outils d'accessibilité.

Ici, l'image représente le produit. Elle est donc significative, et il est utile de la décrire.

Plus tard, vous pourrez remplacer cette *image placeholder* par une vraie image produit.



13. Étape 10 – Ajouter une ligne d'information avec une icône

Nous allons maintenant utiliser **Icon**. Une icône peut aider à repérer rapidement une information.

Ici, nous allons afficher **la note du produit** avec une **icône étoile**.

Pour **aligner l'icône** et le **texte** sur une même ligne, nous utiliserons une **Row**.

Row sera étudié plus en détail dans la séance suivante. Ici, nous l'utilisons simplement pour présenter correctement l'icône et le texte.

Ajoutez ce composable :

```
@Composable  
fun ProductRating(  
    rating: Double,  
    modifier: Modifier = Modifier  
) {  
    Row(modifier = modifier) {  
        Icon(  
            imageVector = Icons.Default.Star,  
            contentDescription = "Note du produit"  
        )  
        Text(  
            text = "$rating / 5",  
            modifier = Modifier.padding(start = 8.dp)  
        )  
    }  
}
```

Puis ajoutez-le dans **ProductDetailScreen**, après la **carte de prix** :

```
ProductRating(  
    rating = product.rating,  
    modifier = Modifier.padding(top = 16.dp)  
)
```

- **Icon** sert à afficher une icône.
- **Icons.Default.Star** représente une étoile.
- Le texte affiche la note sous forme lisible.
- La **contentDescription** de l'icône indique ce qu'elle représente.

A ce moment-là vous voyez **Icons.Default.Star** affiché en rouge dans le code du composant **ProductRating**.

Pour utiliser **Icons.Default.Star** il faut généralement ajouter ces imports :



```
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Star
```

Et il faut aussi avoir la **dépendance** des icônes **Material** dans **build.gradle.kts** !

14. Ajouter la dépendance des icônes Material

Avant d'utiliser **Icon** avec une icône comme **Icons.Default.Star**, il faut vérifier que le projet possède la dépendance des **icônes Material**.

Nous allons profiter de cette étape pour voir **comment ajouter une dépendance** dans un projet Android.

La dépendance doit être ajoutée dans le fichier du module : **app/build.gradle.kts** (et non dans le fichier **build.gradle.kts** situé à la racine du projet !).

Méthode 1 – Ajouter directement la dépendance

Dans le bloc **dependencies{...}**, on peut ajouter directement :

```
implementation("androidx.compose.material:material-icons-extended")
```

Cette méthode est rapide mais Android Studio vous affichera un avertissement en soulignant la ligne en jaune car ce n'est plus la méthode recommandée par Android.

Méthode 2 – Utiliser la Version Catalog

Dans les projets récents, Android Studio utilise souvent un fichier appelé : **gradle/libs.versions.toml**.

Ce fichier permet de déclarer les dépendances avec des noms comme : **implementation(libs.androidx.core.ktx)**, qui ne nécessite généralement plus de préciser la version de dépendance.

L'intérêt est de **centraliser les dépendances** et **leurs versions** dans un seul fichier, au lieu d'écrire directement les coordonnées complètes dans **build.gradle.kts**. Le **Version Catalog** centralise les versions, et le **Compose BOM** peut gérer les versions compatibles de Compose.

Pour utiliser cette méthode, on ajoute d'abord la dépendance dans **gradle/libs.versions.toml**, dans la section **[libraries]** :

```
androidx-compose-material-icons-extended = { module = "androidx.compose.material:material-icons-extended" }
```

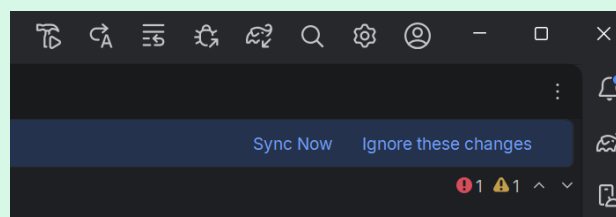
Comme notre projet récent utilise déjà le **Compose BOM**, il n'est généralement pas nécessaire de préciser une version pour les **dépendances Compose** compatibles avec ce BOM.

Ensuite, dans **app/build.gradle.kts**, on peut écrire :

```
implementation(libs.androidx.compose.material.icons.extended)
```

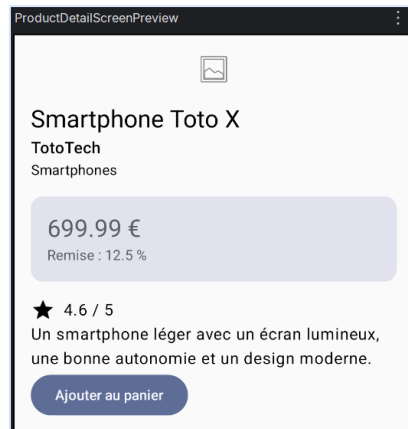
Synchroniser le projet

Après avoir ajouté la dépendance, il faut cliquer sur : « **Sync Now** » dans Android Studio.





Cette synchronisation permet à Android Studio de **télécharger la dépendance** et de reconnaître les imports nécessaires pour utiliser les icônes.



15. Étape 11 – Créer une carte pour les informations de disponibilité

Nous allons maintenant afficher le **stock** et les **informations de livraison** dans une deuxième **Card**.

Cela permet de regrouper des informations proches.

Ajoutez ce composable :

```
@Composable
fun ProductAvailabilityCard(
    stock: Int,
    shippingInformation: String,
    modifier: Modifier = Modifier
) {
    Card(modifier = modifier) {
        Column(
            modifier = Modifier.padding(16.dp)
        ) {
            Text(
                text = "Disponibilité",
                style = MaterialTheme.typography.titleMedium
            )
            Text(
                text = "Stock : $stock unités",
                style = MaterialTheme.typography.bodyMedium
            )
            Text(
                text = shippingInformation,
                style = MaterialTheme.typography.bodyMedium
            )
        }
    }
}
```

Puis ajoutez-le dans **ProductDetailScreen** :

```
ProductAvailabilityCard(
    stock = product.stock,
    shippingInformation = product.shippingInformation,
    modifier = Modifier
        .fillMaxWidth()
        .padding(top = 16.dp)
)
```

Cette carte montre comment regrouper plusieurs textes dans un composant visuel.

→ Elle reçoit les données nécessaires en paramètres.

Disponibilité
Stock : 34 unités
Livraison estimée : 3 à 5 jours ouvrés



→ Elle ne connaît pas tout le produit, seulement ce qu'elle doit afficher.

15. Étape 12 – Créer une section description

La description du produit mérite une section dédiée.

Nous allons créer un composable **ProductDescription**.

Ce composable affichera : un **titre de section** et le texte de **description**.

Ajoutez ce composable :

```
@Composable
fun ProductDescription(
    description: String,
    modifier: Modifier = Modifier
) {
    Column(modifier = modifier) {
        Text(
            text = "Description",
            style = MaterialTheme.typography.titleMedium
        )
        Text(
            text = description,
            style = MaterialTheme.typography.bodyMedium,
            modifier = Modifier.padding(top = 4.dp)
        )
    }
}
```

Puis ajoutez-le dans **ProductDetailScreen** à la place de **Text(text = product.description)** :

```
ProductDescription(
    description = product.description,
    modifier = Modifier.padding(top = 16.dp)
)
```

On continue à découper l'écran en **petits composants**. Cela rend **ProductDetailScreen** plus lisible. Chaque composant a une responsabilité claire.

16. Étape 13 – Ajouter un TextField en lecture seule

Nous allons maintenant manipuler **TextField**. Un **TextField** sert habituellement à saisir ou modifier du texte.

Dans ce TP, nous allons l'utiliser en lecture seule pour afficher une information complémentaire : *la garantie du produit*.

Cela permet de découvrir son apparence sans entrer dans la logique de saisie.

Ajoutez ce composable :

```
@Composable
fun ProductWarrantyField(
    warrantyInformation: String,
    modifier: Modifier = Modifier
) {
    OutlinedTextField(
        value = warrantyInformation,
        onValueChange = {},
        readOnly = true,
        label = {
            Text(text = "Garantie")
        },
        modifier = modifier
    )
}
```



Puis ajoutez-le dans `ProductDetailScreen` :

```
ProductWarrantyField(  
    warrantyInformation = product.warrantyInformation,  
    modifier = Modifier  
        .fillMaxWidth()  
        .padding(top = 16.dp)  
)
```

`OutlinedTextField` est une variante visuelle de `TextField`.

Garantie

Garantie constructeur : 2 ans

Dans le code du composant :

- `value` contient le texte affiché ;
- `onValueChange` est vide ;
- `readOnly = true` indique que le champ n'est pas modifiable ;
- `label` affiche le libellé du champ.

Ce composant est utilisé ici pour comprendre pour l'instant juste **son rôle visuel**.

17. Étape 14 – Ajouter le bouton d'action

Nous allons maintenant isoler le **bouton** dans un composable séparé.

A terme, le **bouton** permettra d'ajouter le produit au panier, mais pour l'instant, le clic ne modifie rien dans l'interface - le composant reçoit simplement un **callback**.

Ajoutez ce composable :

```
@Composable  
fun AddToCartButton(  
    onClick: () -> Unit,  
    modifier: Modifier = Modifier  
) {  
    Button(  
        onClick = onClick,  
        modifier = modifier  
    ) {  
        Text(text = "Ajouter au panier")  
    }  
}
```

Puis remplacez l'ancien bouton dans `ProductDetailScreen` par :

```
AddToCartButton(  
    onClick = onAddToCartClick,  
    modifier = Modifier  
        .fillMaxWidth()  
        .padding(top = 16.dp)  
)
```

- Le **bouton** devient un composant **réutilisable**.
- Il ne décide pas de l'action à effectuer.
- Il signale seulement que l'utilisateur a cliqué.
- Le parent fournit l'action via `onClick`.

18. Étape 15 – Nettoyer ProductDetailScreen

À ce stade, `ProductDetailScreen` doit surtout assembler les composants.

Il ne doit pas contenir toute la logique d'affichage ligne par ligne.

Il doit être facile à lire.



Votre `ProductDetailScreen` peut maintenant ressembler à ceci :

```
@Composable
fun ProductDetailScreen(
    product: ProductUi,
    onAddToCartClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        ProductImage(
            productTitle = product.title,
            modifier = Modifier
                .fillMaxWidth()
                .padding(bottom = 16.dp)
        )

        ProductHeader(
            title = product.title,
            brand = product.brand,
            category = product.category
        )

        ProductPriceCard(
            price = product.price,
            discountPercentage = product.discountPercentage,
            modifier = Modifier
                .fillMaxWidth()
                .padding(top = 16.dp)
        )

        ProductRating(
            rating = product.rating,
            modifier = Modifier.padding(top = 16.dp)
        )

        ProductAvailabilityCard(
            stock = product.stock,
            shippingInformation = product.shippingInformation,
            modifier = Modifier
                .fillMaxWidth()
                .padding(top = 16.dp)
        )

        ProductDescription(
            description = product.description,
            modifier = Modifier.padding(top = 16.dp)
        )

        ProductWarrantyField(
            warrantyInformation = product.warrantyInformation,
            modifier = Modifier
                .fillMaxWidth()
                .padding(top = 16.dp)
        )

        AddToCartButton(
            onClick = onAddToCartClick,
            modifier = Modifier
                .fillMaxWidth()
                .padding(top = 16.dp)
        )
    }
}
```

- L'écran est maintenant composé de plusieurs blocs : *image* · *en-tête* · *prix* · *note* · *disponibilité* · *description* · *garantie* · *bouton*.
- Chaque bloc est un **composable séparé**.
- C'est plus lisible et plus facile à **faire évoluer**.

19. Étape 16 – Ajouter une deuxième *preview*

Une seule **preview** ne suffit pas toujours. Nous allons alors créer un **deuxième produit** pour vérifier que nos composants fonctionnent avec d'autres données.



Ajoutez cette fonction :

```
fun sampleProductOutOfStock(): ProductUi {
    return ProductUi(
        title = "Casque Audio Pulse",
        brand = "SoundPeak",
        category = "Audio",
        description = "Un casque confortable conçu pour écouter de la musique, " +
            "suivre des cours en ligne et travailler dans de bonnes conditions.",
        price = 129.99,
        discountPercentage = 8.0,
        rating = 4.2,
        stock = 0,
        warrantyInformation = "Garantie constructeur : 1 an",
        shippingInformation = "Produit temporairement indisponible"
    )
}
```

Puis ajoutez une deuxième **preview** :

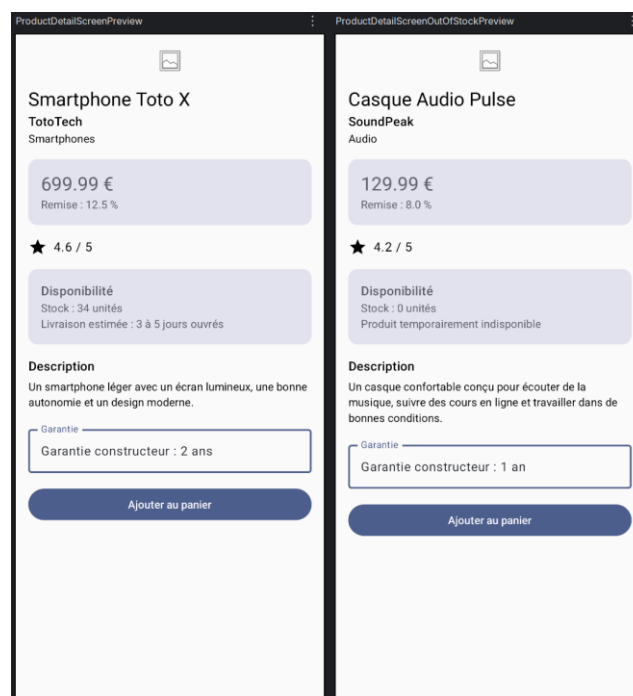
```
@Preview(showBackground = true)
@Composable
fun ProductDetailScreenOutOfStockPreview() {
    ProductExplorerTheme {
        ProductDetailScreen(
            product = sampleProductOutOfStock(),
            onAddToCartClick = {}
        )
    }
}
```

Cette **preview** affiche un autre produit.

Elle permet de vérifier que l'écran reste correct avec :

- un autre **titre** ;
- une autre **catégorie** ;
- un autre **prix** ;
- un stock à **zéro** ;
- une **description** plus longue.

Les **preview**s servent à tester visuellement **plusieurs situations**.





20. Code complet indicatif

Voici une version complète possible de **MainActivity.kt** :

<https://gist.github.com/ariatsi/801573ec3914a3d78483dc9e92adef41> .

Adaptez le nom du package et le nom du thème à votre projet.

Et voici le répertoire GitHub avec cette version :

<https://github.com/ariatsi/ProductExplorer/releases/tag/tp4-final>

21. Ce qu'il faut retenir

Dans ce TP, vous avez démarré l'application **Product Explorer**.

Vous avez construit un premier écran de détail produit avec les composants fondamentaux de Compose.

Vous avez utilisé :

- **Text** pour afficher des informations ;
- **Button** pour représenter une action ;
- **Icon** pour accompagner une information visuelle ;
- **Image** pour afficher une image produit ;
- **Card** pour regrouper des informations ;
- **TextField** pour afficher une information dans un champ ;
- **Preview** pour vérifier plusieurs rendus.

Vous avez aussi continué à appliquer les bonnes pratiques vues précédemment :

- **découper** l'interface en petits composables ;
- **passer** les données en **paramètres** ;
- **utiliser modifier**: **Modifier = Modifier** ;
- **laisser** le parent décider du placement **externe** ;
- **utiliser** des **callbacks** pour les **actions** ;
- **penser à l'accessibilité** avec **contentDescription**.

Dans le prochain TP, vous conserverez cette base et vous construirez un écran d'accueil pour Product Explorer.